Class Files

For some assignments you may be provided with an enemy agent. The enemy agent will be a *.class* file that you will need to add to your class path.

First create an empty folder to hold the provided .class files. Call it something like "enemy_agents". Then you will need to create the package directory structure for the .class file. If the .class expects to be in package *edu.cwru.sepia.agent* then underneath your "enemy_agents" folder you need to have the folder hierarchy *edu/cwru/sepia/agent*. Then in the agent folder you will put the provided class file.

Next you have to add the enemy_agent folder to the classpath. Right click on the project and select *Build Path->Configure Build Path*. Then Click the button that says "Add class folder". Select the enemy_agents folder and save. You are now ready to use the class. file.

If you get errors about the class not being found double check that the package and agent name in the config file match the folder heirarchy and class name you just set up.

Command Line Setup

It is also possible to avoid Eclipse and use the command line. Compiling and running use the standard *javac* and *java* commands respectively. The most important thing to consider is the directory you run SEPIA from and the classpath arguments you use.

Consider a directory structure like the following

```
MySEPIAProject
  - data
    - my_map.xml
    - my_config.xml
  - lib
    - Sepia.jar
  - src
    - edu
      - cwru
        - sepia
          - agent
            MyAgent.java
```

To compile this you would go into the *MySEPIAProject* folder and run the following

```
javac -cp lib/Sepia.jar src/edu/cwru/sepia/agent/MyAgent.java
```

This will produce a *MyAgent.class* file in the *src/edu/cwru/sepia/agent* directory. To run the configuration with this new class file you would use the command

```
java -cp lib/Sepia.jar:src edu.cwru.sepia.Main2 data/my_config.xml
```

Note

On Windows the classpath arguments will be separated by semicolons, whereas on Unix based systems (Linux, Mac, etc) they will be separated by a colon.

Note

If you get a strange error about JAXBException not being found when trying to start the jvm, please see this Stackoverflow thread.

Class Files

If you are provided with a .class file first create a new folder to act as the root of the provided class files. Call it something like "enemy_agents". Then create the folder hierarchy to match the class file's expected package. For instance if the expected package is *edu.cwru.sepia.agent* then you will need to put the folder hierarchy *edu/cwru/sepia/agent* underneath your new enemy_agents folder. Then you will place the class file in the agent folder.

To run you will need to include the path to the enemy_agents folder in your class path. For example:

```
java -cp lib/Sepia.jar:enemy_agents:src edu.cwru.sepia.Main2 data/my_config.xml
```

If you have issues double check the folder hierarchy matches the path and that the config file specifies that package and the proper class file name.

Running Sepia

This section covers how to specify scenario configurations and how to use Eclipse to run a scenario.

SEPIA Working Directory

All of the directories specified in SEPIA configuration files or on the command line are relative to the current working directory. For example if you are in the directory "mydir" and the configuration file specifies the map "data/map.xml" then SEPIA will look for the map in mydir/data/map.xml.

If SEPIA is complaining about missing maps, configurations or agents please double check which directory you are running the application from. Next make sure that your class path variable includes the SEPIA.jar file and any folder structure containing .class files. Finally check all configurations to make sure that file locations are specified relative to the directory SEPIA is run from.

Adding a Map

In order to begin testing our agent we will need a test map and a test configuration. First create a data folder in your project in the same way we created the lib folder when setting up the project. Then click here to download the example map Extract the files to the new data folder.

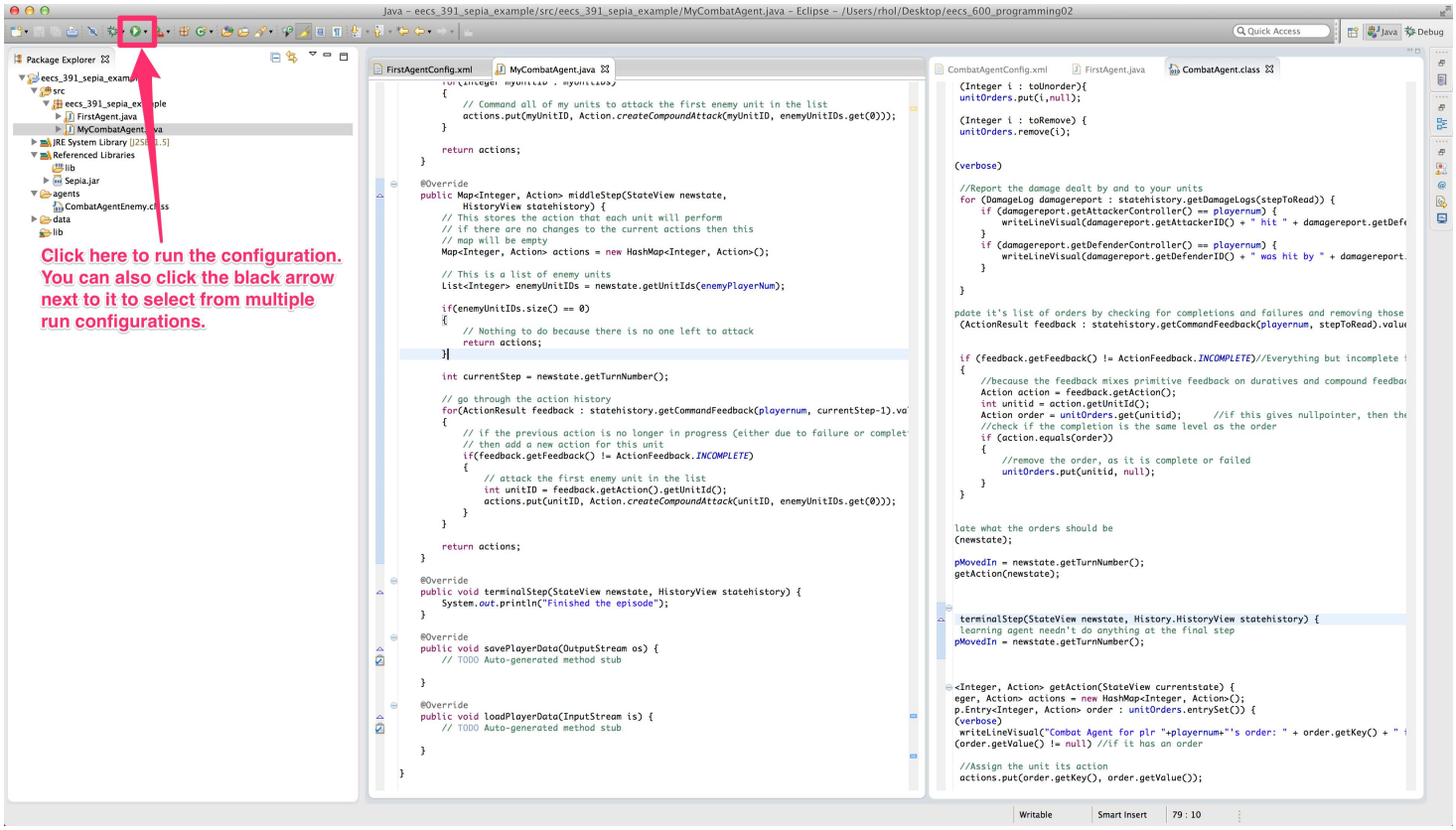
Open up the `ResourceCollectionConfig.xml` file. This XML file specifies the map, the agents used, the goals of the scenario and extra parameters about how many episodes of the scenario to run before quitting.

The most important tag is the `<Player>` tag. The ID attribute specifies which player this agent is controlling. This is the same number passed into the constructor of your class. Also look at the `<ClassName>` tag. This specifies the Java class path to the agent class that controls the player.

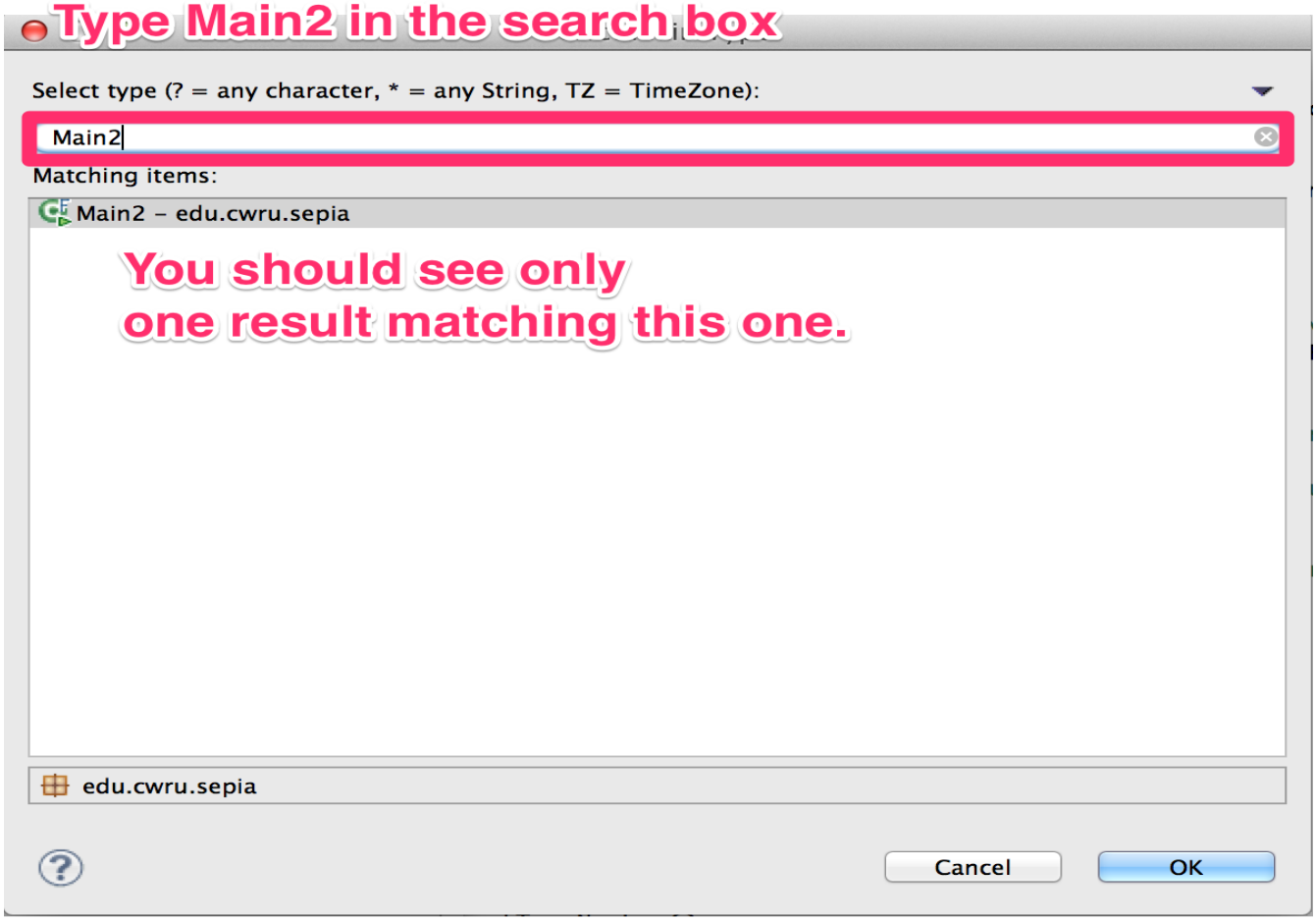
You'll also notice that player 0 has two agents: VisualAgent and ResourceCollectionAgent. ResourceCollectionAgent comes with Sepia as an example. It contains an agent that controls a few peasant units to collect some gold and wood. VisualAgent is a special agent. It will open a Java Swing window that gives a visual representation of the game. There are two argument parameters to the VisualAgent. Setting the second one to true will allow you to control the units of the VisualAgent's PlayerID.

Running Resource Collection Agent Example

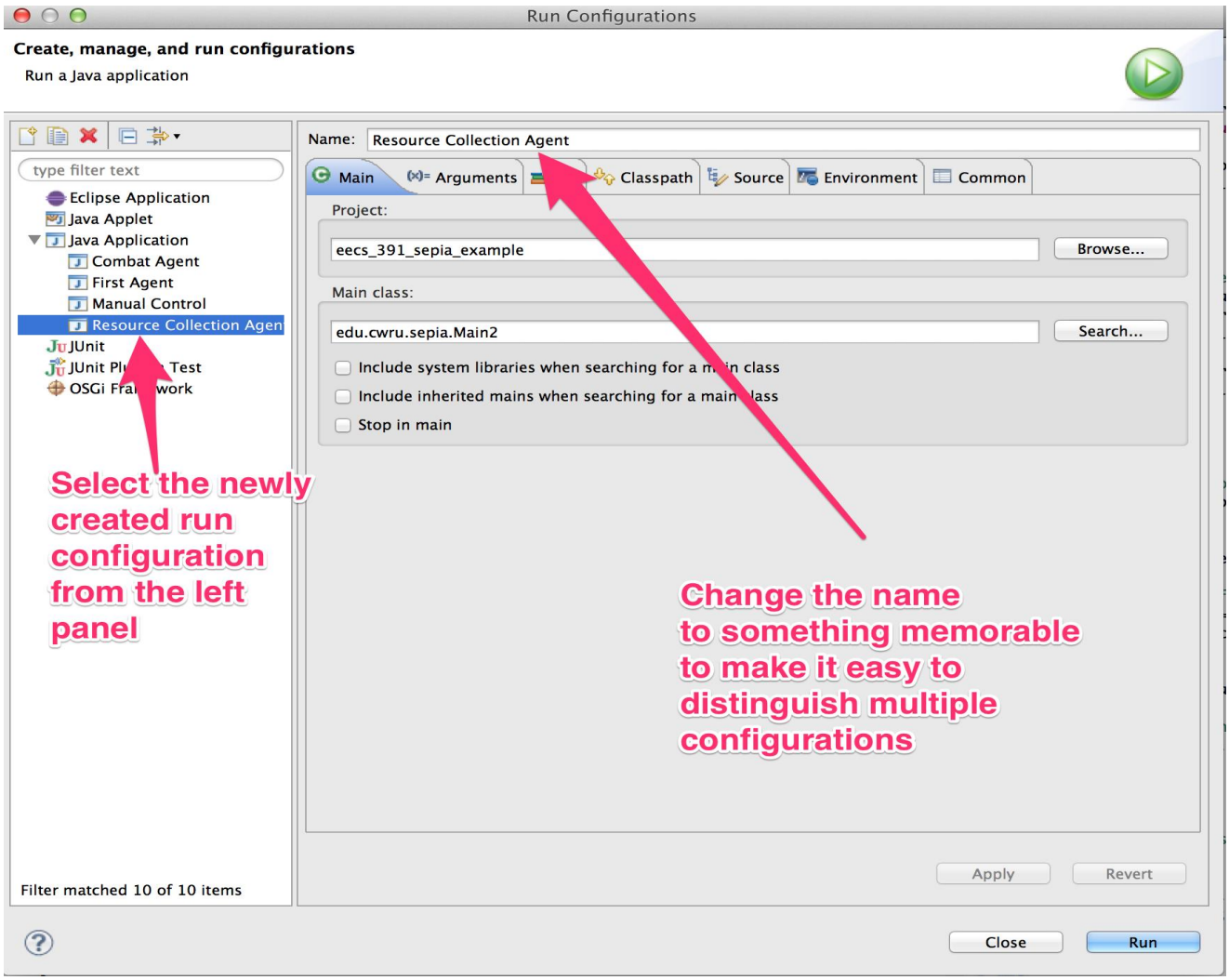
To run SEPIA first highlight the `Sepia.jar` file in the Package Explorer. It should be under the `Referenced Libraries` folder. Then click the green Run arrow on the menu bar



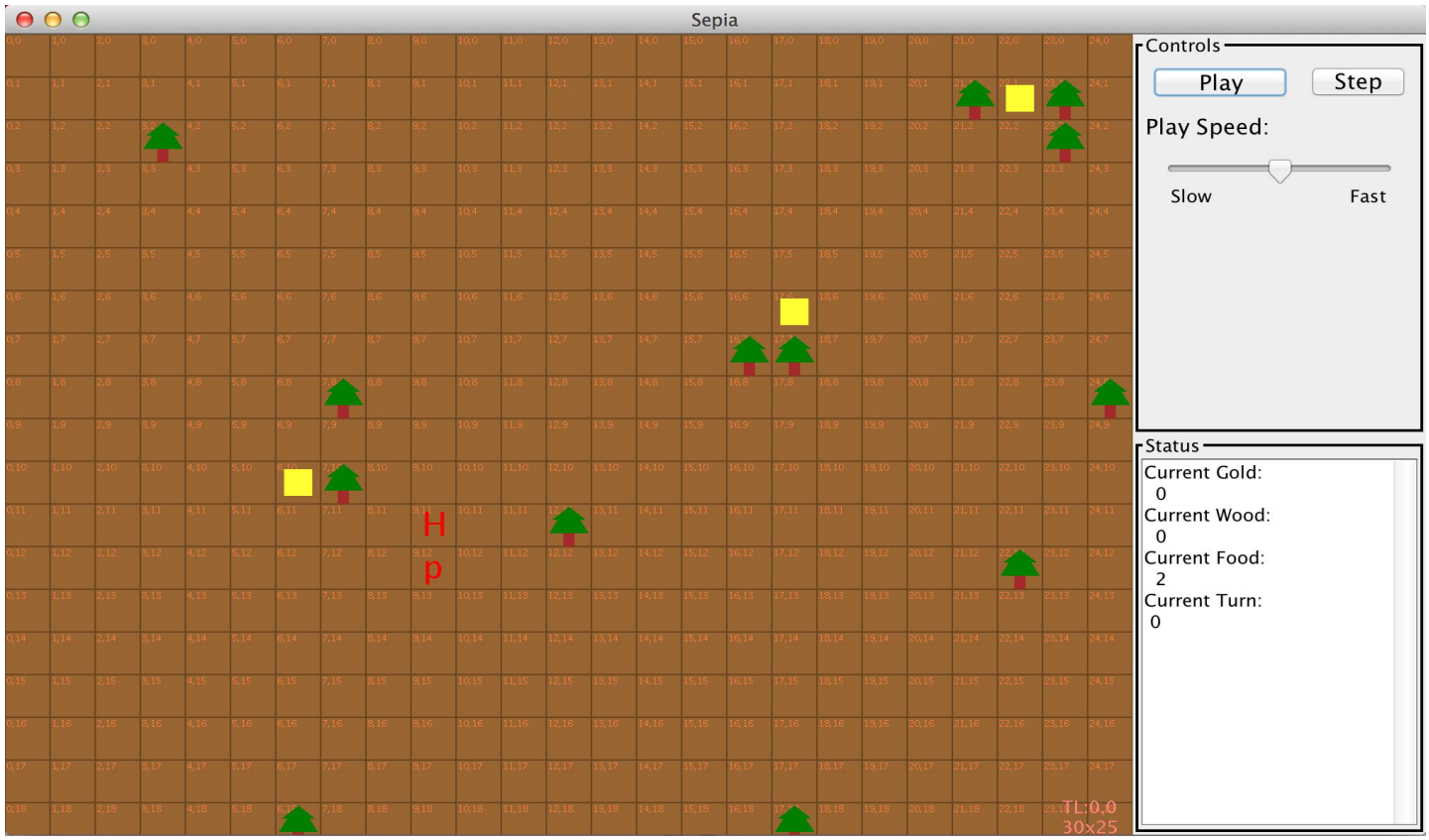
This will open a dialog box. Select `Java Application` and then click ok. Next you will see a box with a few of the built in SEPIA Classes. Select the `Main2` class.



This should open a Console tab at the bottom of eclipse that will complain about no configuration file being specified. To fix this click `Run->Run Configurations` in the main menu. This will allow you to edit the run parameters to specify things like command line arguments and working directories. Under the `Java Applications` category on the left there should be a `Main2`. Click on that to edit the run configuration.



First give this a useful name. Change Main2 to something like "Resource Collection Agent". Then click on the arguments tab and in the `Program Arguments` box type "data/ResourceCollectionConfig.xml". Click the `Apply Button` and then click the `Run Button`. This time a window should open up.



This window is the Visual Agent. You'll see yellow squares, trees and some letters. The yellow squares are gold mines, the trees are wood sources. The P is a peasant unit and the H is the Town Hall unit. Click on the `Play Button` to run the scenario and see the agent in action. You should see the Peasant begin moving to the gold mines and trees. If you look on the right side of the screen you can see information about how much gold, wood and food you currently have.

Note

The food number is actually the amount of food in use. There is a food supply cap in the game code (which starts at 4 in this scenario). Agents can create units that use food up until the supply cap. Unfortunately as of this version of SEPIA nothing displays this supply cap. However, you can access the supply cap amount in the agent code.

Manual Control

Now lets create an alternative run configuration for the VisualAgent so you can manually control the peasants. Again using the main menu click `Run->Run Configuration...`. Find the `Resource Collection Agent` configuration then click on the duplicate button. Change the name to something meaningful (in this case "Manual Control"). Then in the `Arguments` tab change the value of the `Program Arguments` to "data/ManualControlConfig.xml". Finally click `Apply` and then `Run`. This time if you click play nothing will happen. First left click on the peasant to highlight it. Then right click on one of the gold mines and select "Gather GOLD_MINE". Now if you click play the Peasant should move to that gold mine and begin collecting gold. Double clicking on the peasant will show its unit id, its health and what resources it is carrying. You can control any of the units in this way in order to manually try things out in the game.

Configuration File

The configuration file specifies the parameters of a SEPIA scenario. This includes specifying which map to use, which agents to run and the termination conditions for the scenario.

Open `data/ResourceCollectionConfig.xml` file in Eclipse to follow along. Each configuration file has the root tag of `<Configuration>`. All other tags are children of this tag.

Use the `<Map>` tag to specify the location of the map file with respect to the directory you will be running SEPIA from. In this example the working directory is the main project directory and the maps are all under the data folder. So the map name is prepended with `data/`.

Use the `<Player>` tag to specify agents. The Player tag has a required attribute `Id`. This should be set to an integer that is the player number. In general there will be two players on the map. Your player will have playernumber 0 and the enemy will have playernumber 1.

Inside the Player tag is the `<AgentClass>` tag. This class specifies which Java class will control this player number's units. Inside the AgentClass tag you must specify the `<ClassName>` tag. The text of this tag is the fully qualified name of the Java class. So if you have a .class file in the subfolders `edu/cwru/sepia/agent` then the text will be `edu/cwru/sepia/agent/MyAgent.class`. If you had your class file in the top directory it would be `MyAgent.class`.

In addition to the required ClassName tag you can optionally specify as many `<Argument>` tags as you want. The text of each of these tags is passed into the agent's constructor as an array of Strings. So you will need to convert the strings to the proper data types.

There are two more sections of interest in the config file. The `<ModelParameters>` and the `<Runner>`. The ModelParameters tag specifies the termination conditions of the scenario. The Runner specifies which game engine is used. For the purpose of this tutorial and assignments you should not need to modify these.

Visual Agent

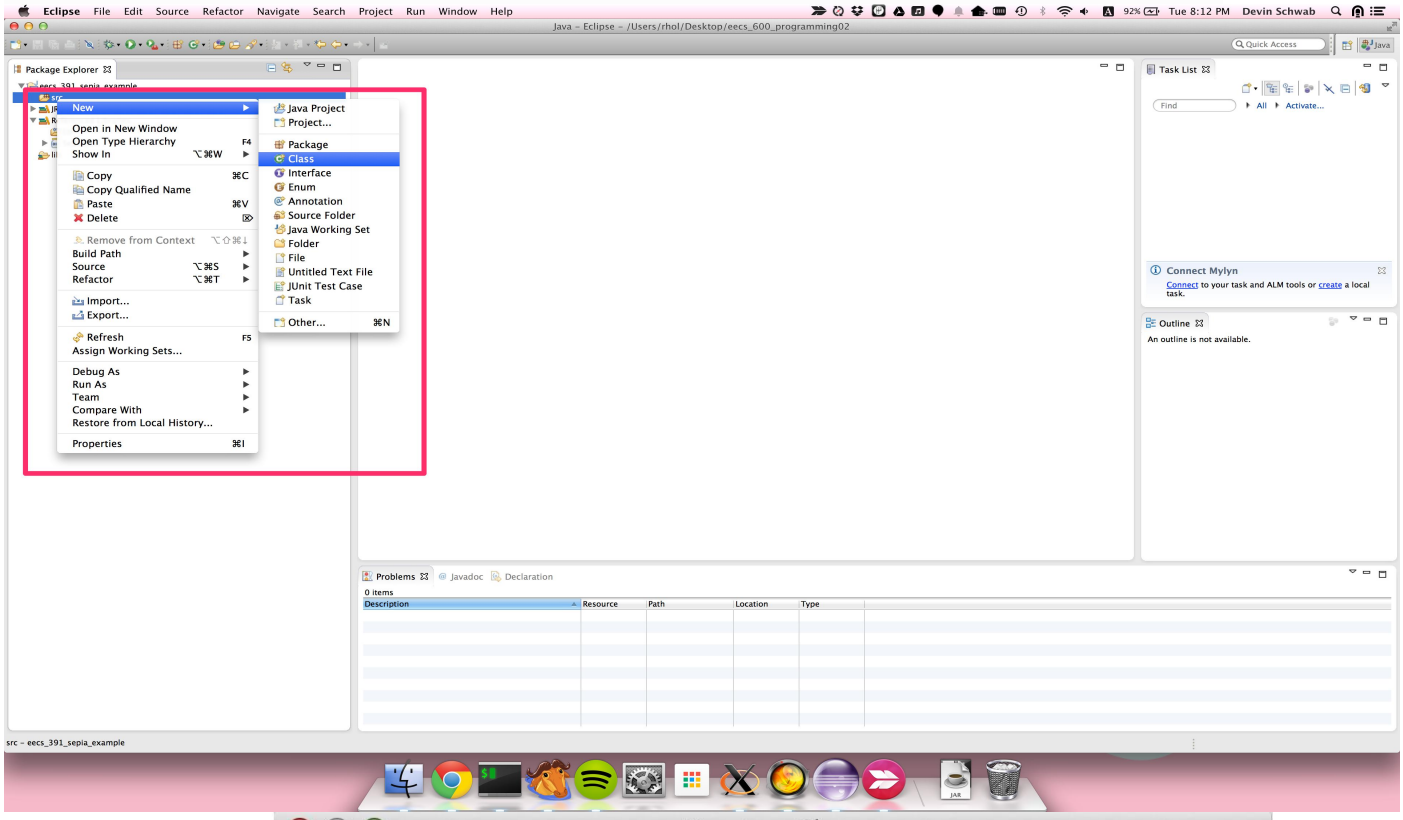
You may have noticed that in the provided configuration files there are two Player tags with the same Id. One of these player tags specifies that a class called VisualAgent is used. VisualAgent is a convenient way of visualizing what is happening in a SEPIA scenario. You can use it for both manual control as well as visual debugging of an agent's behavior.

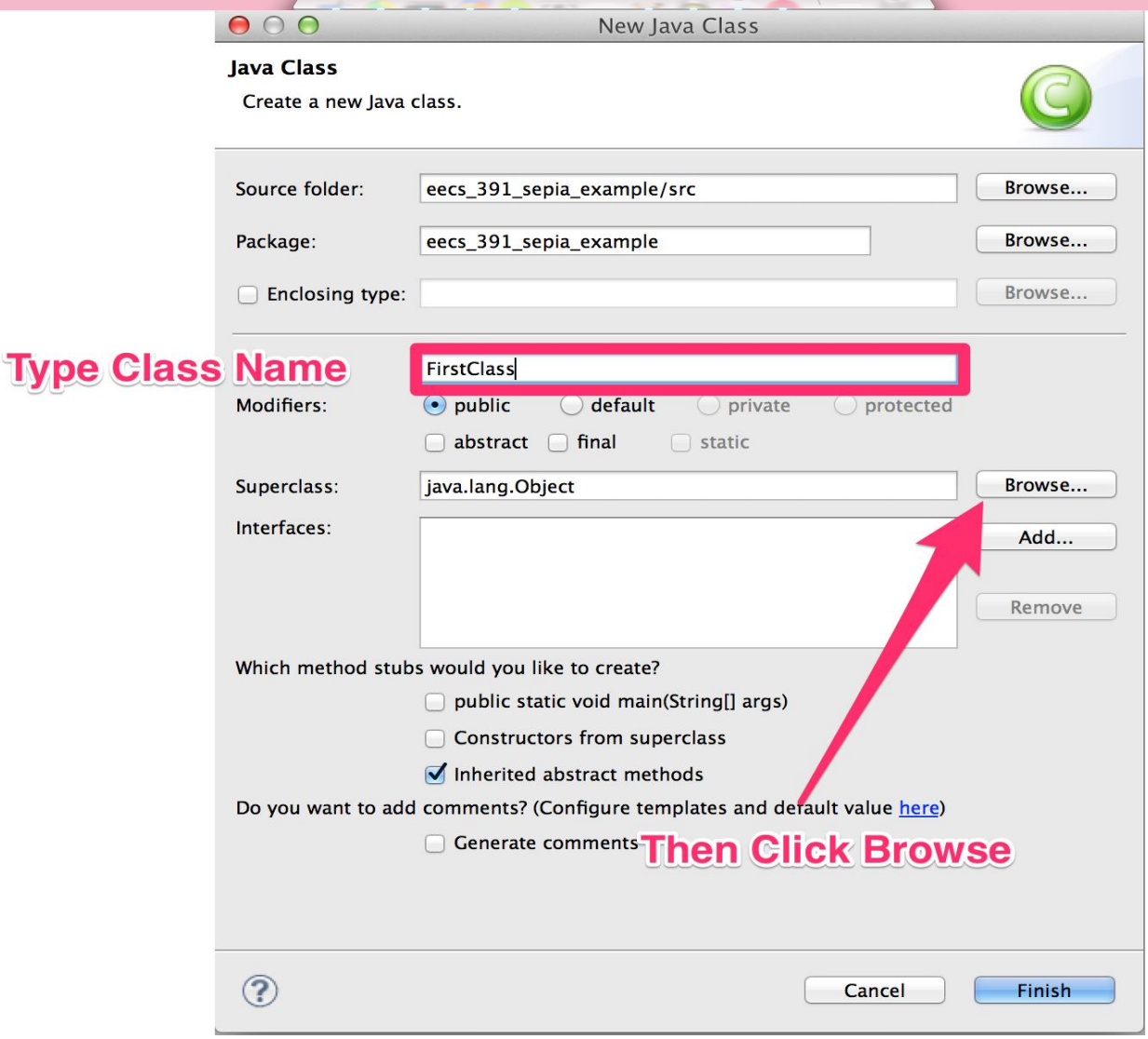
While VisualAgent is useful for writing your agent, when you want to actually collect data or automate the testing the VisualAgent can be a hindrance. To run without the VisualAgent simply comment its player tag out of the configuration file. When you run without the visual agent the scenario will automatically run at full speed until the scenarios are over.

Your First Agent

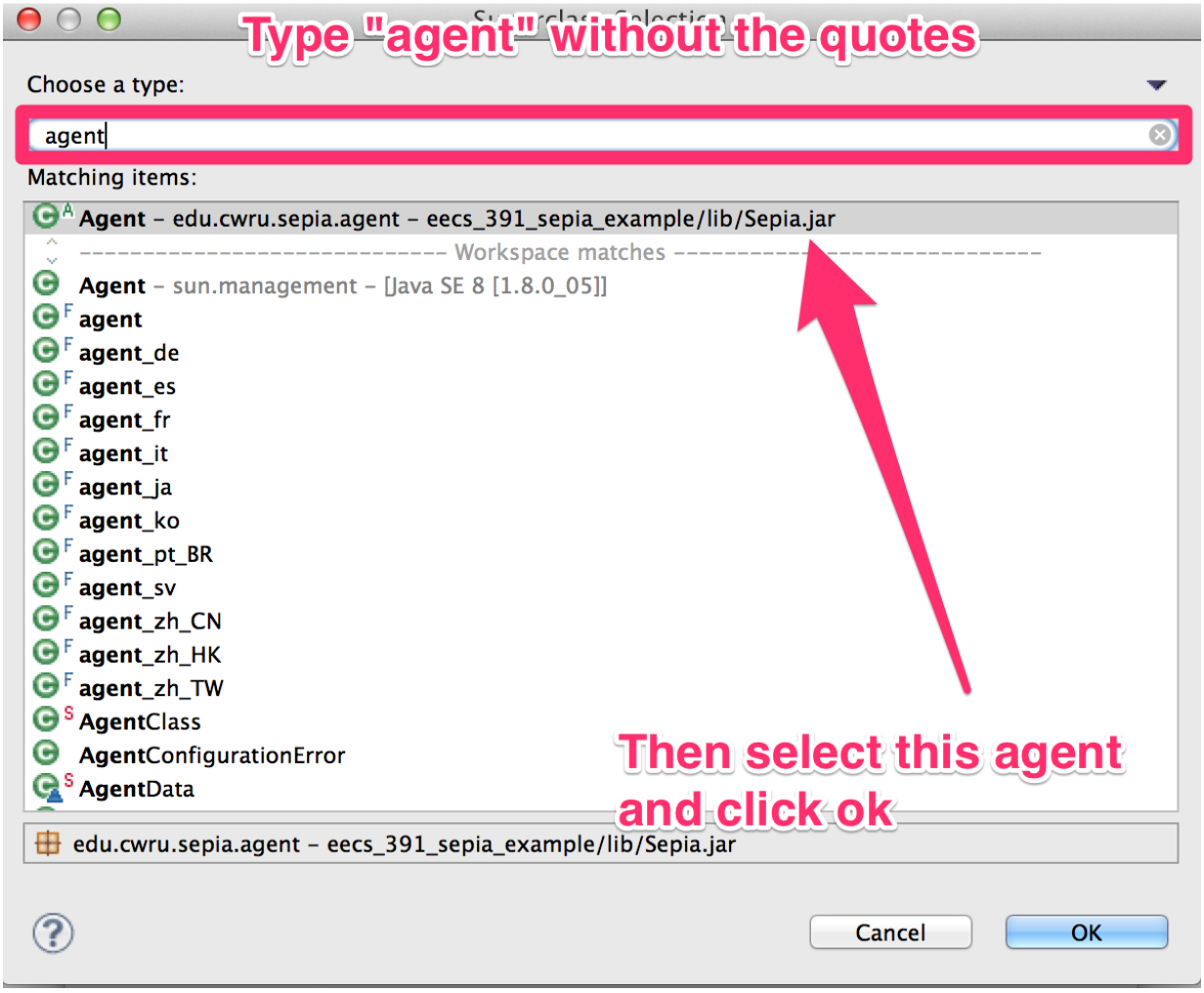
Create the Class

All agents in Sepia are subclass of the `edu.cwru.sepia.agent.Agent` class. To create the class skeleton which we will fill in right click on the `src` fold and select `New->Class`. This will open a new window where you will specify the name and the super class.

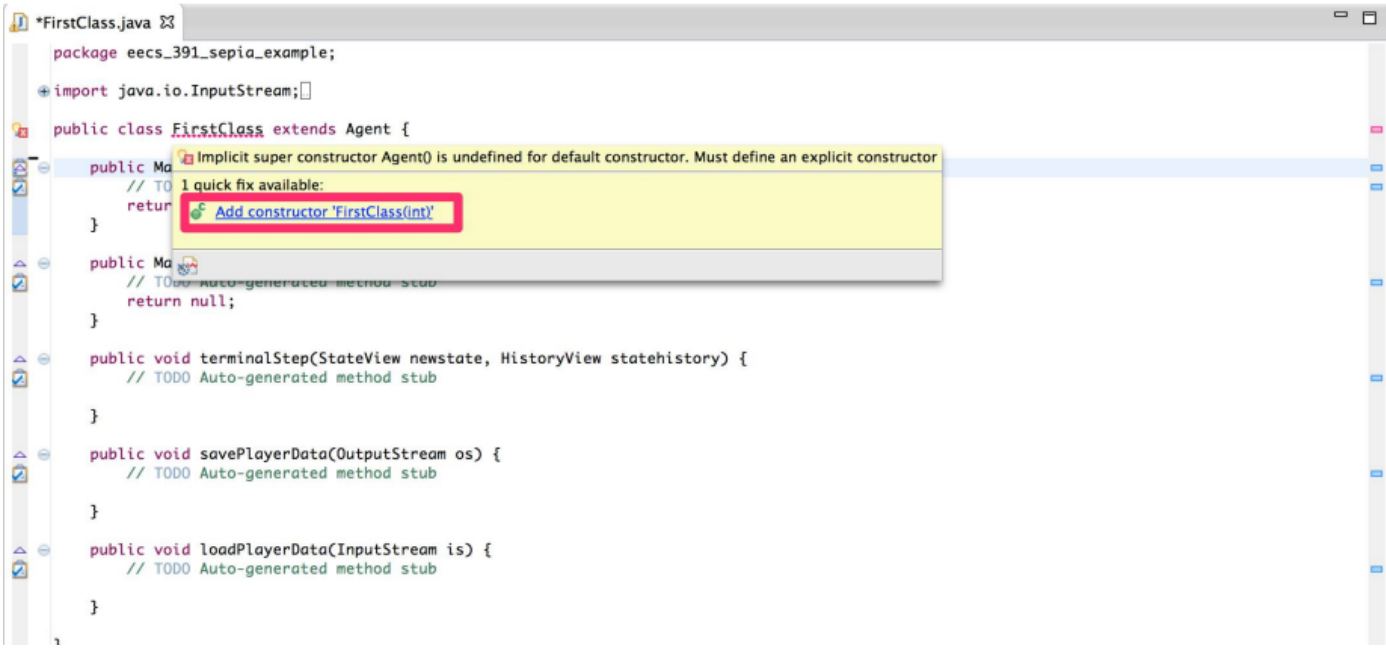




In the `Name` field put in the name of your class. In this tutorial we will be using the name `FirstClass`. Then click the `Browse` button next to the super class line. This will open a new window. In the search line type the word "agent" without the quotes. You should see a bunch of results and the top one should say `edu.cwru.sepia.agent.Agent`. Click on this line and then click the ok button. You can now close the new class window by clicking Finish.



You should now have an empty class with five blank methods. Before we run our agent we will need to fill in these methods. First thing is that your inherited class needs a constructor that takes in an integer. This integer specifies the instance's player number which is important for determining who units belong to. Insert the default constructor by mousing over the class name and clicking on the **Add constructor** option. You should now have a class that compiles.



Setup the Configuration

Open the `FirstAgentConfig.xml` file from the data file. Change the text of the `<ClassName>` tag to match the full class path of your agent. Save the file.

Now make a duplicate of one of the run configurations from the *Running Resource Collection Agent Example* section. Change the configuration file to `FirstAgentConfig.xml`. You can run this configuration now, but your agent will not do anything.

Basic Agent API

There are three main methods you need to implement in your agent: `initialStep`, `middleStep`, and `terminalStep`. The `initialStep` method is run once before the start of an episode. This is a good method to do any special initialization that requires information about the map and your units. You also specify your first unit actions in this method. The `middleStep` is called every other round during an episode. This is where the bulk of your agent logic will go. You will be able to look at the current state of the game, the history of the game and assign actions to your units. Finally `terminalStep` is run once after an episode has finished. You cannot specify anymore unit actions as the episode is over, but you can look at the final state and the game history. This is a good place to sum up any statistics and log them.

Each of these methods is passed two objects: a StateView and a HistoryView object. StateView contains all of thei nformation about the current state of the game. Normally this includes the entire game state, but if fog of war is enabled then you will only be able to see information about units near your units. HistoryView contains information about events that have occurred on previous turns such as HP changes and unit deaths and creations.

You use the StateView object to access information about your units such as their type, ID, location, HP and held resources. You will use this information to construct actions to put in the action map. To get a list of all of your units you will use the `stateview.getUnitIds(playernum)` where `playernum` is your player number or the enemy you are getting information about. Your agent can access its playernum via the superclass's playernum variable. This method will return a list of Integers. To find out what type of unit corresponds to each unit ID you will need to get the UnitView and then the TemplateView. To get a UnitView given an ID use the method `stateview.getUnit(unitID)`. To get the TemplateView of a UnitView use the method `unitView.getTemplateView()`. The TemplateView allows you to view default information for given unit types. Here is a small code snippet illustrating how to access all units for an agent and print their types.

```
List<Integer> unitIDs = stateview.getUnitIds(playernum);

for(Integer unitID : unitIDs)
{
  UnitView unitView = stateview.getUnit(unitID);

  TemplateView templateView = unitView.getTemplateView();

  System.out.println(templateView.getName() + ": " + unitID);
}
```

Useful UnitView Methods

All of these methods are called on instances of UnitView.

getCargoAmount()
Returns the numerical amount of the resource that unit is holding

getCargoType()
Returns the type of resource the unit is holding (either Gold, Wood, or nothing)

getHP()
Numerical amount of health remaining. When the unit health reaches 0 it will die

getXPosition()
X position in the map (starts at 0 with 0 on the left side)

getYPosition()
Y position in the map (starts at 0 with 0 at the top)

getCurrentDurativeAction()
Returns the current action the unit is performing or none.

getCurrentDurativeProgress()
Returns the numerical progress towards completion. The higher the number the closer to completion.

Useful Map Methods

All of these methods are called on a StateView object

getXExtent()
The width of the map

getYExtent()
The height of the map

getResourceNodeIds(<Resource Type>)
Gets a list of resource node ids. This will allow you to access the resource nodes on the map. Similar to a UnitView a resource node object will allow you to find information about the resources such as position and amount remaining. <Resource Type> can be either `Type.GOLD_MINES` or `Type.TREE` to get a list of Gold Mines and a list of Trees respectively.

Once you have a list of Resource IDs you can get resource nodes with the method `stateview.getResourceNode(<ID>)`. Then you can access information about the resource with the following methods.

resource.getXPosition()
X Position on the map of the resource

resourcre.getYPosition()
Y Position on the map of the resource

resource.getAmountRemaining()
Numerical amount of the resource remaining in the resource. Once this number reaches 0 the resource will disappear from the map

Compound Action Descriptions

To create a compound action you will need to construct an appropriately typed action. Each of the descriptions lists the proper action type. Here is a code snippet demonstrating the construction of a compound gather action

```
Action gatherAction = new TargetedAction(pesantID, ActionType.COMPOUNDGATHER, goldMineID);
```

COMPOUNDATTACK

- targeted attack
- parameters: your unit ID and enemy unit ID
- moves into range of the unit and attacks it once

COMPOUNDBUILD

- Located production action
- parameters: your unit ID, building template ID, X and Y
- moves to the specified location and then builds until the building is done

COMPOUNDDEPOSIT

- targeted action
- parameters: your unit ID and a town hall ID
- moves next to the town hall and deposits all held resources

COMPOUNDGATHER

- targeted action
- parameters: your unit ID and the ID of the resource node
- moves next to specified resource and gathers as many resources as can be held

COMPOUNDMOVE

- located action
- parametesr: your unit ID, X and Y
- moves to the specified location

COMPOUNDPRODUCE

- production action
- parameters: building ID and unit template ID
- does a primitive produce until the unit is complete

Primitive Action Descriptions

To create a primitive action you will use the factory methods in the Action class. For example:

```
Action primitiveProduce = Action.createPrimitiveProduce(buildingID, templateID);
```

PRIMITIVEATTACK

- TargetedAction
- Parameters are your unit's ID and the enemy unit's ID
- Tries to attack the enemy unit. If it's out of range, nothing happens.

PRIMITIVEBUILD

- ProductionAction
- Parameters are your unit's ID and the building template ID.
- Does one turn of work towards building that building on your unit's spot.
- When the building finishes, your unit will move off to one side.

PRIMITIVEDEPOSIT

- DirectedAction
- Parameters are your unit's ID and a direction (Direction enum, i.e. Direction.NORTH)
- Looks in the direction provided and if there's a town hall, deposits resources there.

PRIMITIVEGATHER

- DirectedAction
- Parameters are your unit's ID and a direction
- Looks in that direction and tries to gather resources from it.

PRIMITIVEMOVE

- DirectedAction
- Parameters are your unit's ID and a direction
- Tries to move in that direction one step. If there's something there, nothing happens.

PRIMITIVEPRODUCE

- ProductionAction
- Parameters are your building's ID and the template ID for the unit.
- Does one turn's work towards creating the unit specified.

Moving Your Units

Here is the code for an agent that takes each peasant and moves it repeatedly left. While this agent isn't very useful it does provide an example of the basic API including how to find your unit IDs and how to command your units.

```
public Map middleStep(StateView newstate, HistoryView statehistory) {
        // This stores the action that each unit will perform
        // if there are no changes to the current actions then this
        // map will be empty.
        Map<Integer, Action> actions = new HashMap<Integer, Action>();

        // this will return a list of all of your units
        // You will need to check each unit ID to determine the unit's type
        List<Integer> myUnitIds = newstate.getUnitIds(playernum);

        // These will store the Unit IDs that are peasants and townhalls respectively
        List<Integer> peasantIds = new ArrayList<Integer>();
        List<Integer> townhallIds = new ArrayList<Integer>();

        // This loop will examine each of our unit IDs and classify them as either
        // a Townhall or a Peasant
        for(Integer unitID : myUnitIds)
        {
                // UnitViews extract information about a specified unit id
                // from the current state. Using a unit view you can determine
                // the type of the unit with the given ID as well as other information
                // such as health and resources carried.
                UnitView unit = newstate.getUnit(unitID);

                // To find properties that all units of a given type share
                // access the UnitTemplateView using the `getTemplateView()`
                // method of a UnitView instance. In this case we are getting
                // the type name so that we can classify our units as Peasants and Townhalls
                String unitTypeName = unit.getTemplateView().getName();

                if(unitTypeName.equals("TownHall"))
                        townhallIds.add(unitID);
                else if(unitTypeName.equals("Peasant"))
                        peasantIds.add(unitID);
                else
                        System.err.println("Unexpected Unit type: " + unitTypeName);
        }

        // Now that we know the unit types we can assign our peasants to collect resources
        for(Integer peasantID : peasantIds)
        {
                actions.put(peasantID, Action.createPrimitiveMove(peasantID, Direction.EAST));
        }

        return actions;
}
```

Collecting Resources

Here is the code for an agent that commands the default peasant to alternate collecting wood and gold. Click here to download this version of the agent

```
public Map initialStep(StateView newstate, HistoryView statehistory) {
	return middleStep(newstate, statehistory);
}

public Map middleStep(StateView newstate, HistoryView statehistory) {
	// This stores the action that each unit will perform
	// if there are no changes to the current actions then this
	// map will be empty.
	Map<Integer, Action> actions = new HashMap<Integer, Action>();

	// this will return a list of all of your units
	// You will need to check each unit ID to determine the unit's type
	List<Integer> myUnitIds = newstate.getUnitIds(playernum);

	// These will store the Unit IDs that are peasants and townhalls respectively
	List<Integer> peasantIds = new ArrayList<Integer>();
	List<Integer> townhallIds = new ArrayList<Integer>();

	// This loop will examine each of our unit IDs and classify them as either
	// a Townhall or a Peasant
	for(Integer unitID : myUnitIds)
	{
		// UnitViews extract information about a specified unit id
		// from the current state. Using a unit view you can determine
		// the type of the unit with the given ID as well as other information
		// such as health and resources carried.
		UnitView unit = newstate.getUnit(unitID);

		// To find properties that all units of a given type share
		// access the UnitTemplateView using the `getTemplateView()`
		// method of a UnitView instance. In this case we are getting
		// the type name so that we can classify our units as Peasants and Townhalls
		String unitTypeName = unit.getTemplateView().getName();

		if(unitTypeName.equals("TownHall"))
			townhallIds.add(unitID);
		else if(unitTypeName.equals("Peasant"))
			peasantIds.add(unitID);
		else
			System.err.println("Unexpected Unit type: " + unitTypeName);
	}

	// get the amount of wood and gold you have in your Town Hall
	int currentGold = newstate.getResourceAmount(playernum, ResourceType.GOLD);
	int currentWood = newstate.getResourceAmount(playernum, ResourceType.WOOD);

	List<Integer> goldMines = newstate.getResourceNodeIds(Type.GOLD_MINE);
	List<Integer> trees = newstate.getResourceNodeIds(Type.TREE);

	// Now that we know the unit types we can assign our peasants to collect resources
	for(Integer peasantID : peasantIds)
	{
		Action action = null;
		if(newstate.getUnit(peasantID).getCargoAmount() > 0)
		{
			// If the agent is carrying cargo then command it to deposit what its carrying at the townhall.
			// Here we are constructing a new TargetedAction. The first parameter is the unit being commanded.
			// The second parameter is the action type, in this case a COMPOUNDDEPOSIT. The actions starting
			// with COMPOUND are convenience actions made up of multiple move actions and another final action
			// in this case DEPOSIT. The moves are determined using A* planning to the location of the unit
			// specified by the 3rd argument of the constructor.
			action = new TargetedAction(peasantID, ActionType.COMPOUNDDEPOSIT, townhallIds.get(0));
		}
		else
		{
			// If the agent isn't carrying anything instruct it to go collect either gold or wood
			// whichever you have less of
			if(currentGold < currentWood)
			{
				action = new TargetedAction(peasantID, ActionType.COMPOUNDGATHER, goldMines.get(0));
			}
			else
			{
				action = new TargetedAction(peasantID, ActionType.COMPOUNDGATHER, trees.get(0));
			}
		}

		// Put the actions in the action map.
		// Without this step your agent will do nothing.
		actions.put(peasantID, action);
	}

	return actions;
}

public void terminalStep(StateView newstate, HistoryView statehistory) {
	System.out.println("Finsihed the episode");
}
```

Running this class should print out "Constructed My First Agent" in the console when SEPIA starts.

Clicking play will cause the single peasant to alternate between collecting gold and wood.

This version of agent:

```
package eecs_391_sepia_example;

import java.io.InputStream;
import java.io.OutputStream;
import java.util.ArrayList;
import java.util.HashMap;
import java.util.List;
import java.util.Map;

import edu.cwru.sepia.action.Action;
import edu.cwru.sepia.action.ActionType;
import edu.cwru.sepia.action.TargetedAction;
import edu.cwru.sepia.agent.Agent;
import edu.cwru.sepia.environment.model.history.History.HistoryView;
import edu.cwru.sepia.environment.model.state.ResourceNode.Type;
import edu.cwru.sepia.environment.model.state.ResourceType;
import edu.cwru.sepia.environment.model.state.State.StateView;
import edu.cwru.sepia.environment.model.state.Unit.UnitView;

public class FirstAgent extends Agent {

 private static final long serialVersionUID = -7481143097108592969L;

 public FirstAgent(int playernum) {
  super(playernum);

  System.out.println("Constructed My First Agent");
 }

 public Map initialStep(StateView newstate, HistoryView statehistory) {
  return middleStep(newstate, statehistory);
 }

 public Map middleStep(StateView newstate, HistoryView statehistory) {
  // This stores the action that each unit will perform
  // if there are no changes to the current actions then this
  // map will be empty.
  Map<Integer, Action> actions = new HashMap<Integer, Action>();

  // this will return a list of all of your units
  // You will need to check each unit ID to determine the unit's type
  List<Integer> myUnitIds = newstate.getUnitIds(playernum);

  // These will store the Unit IDs that are peasants and townhalls respectively
  List<Integer> peasantIds = new ArrayList<Integer>();
  List<Integer> townhallIds = new ArrayList<Integer>();

  // This loop will examine each of our unit IDs and classify them as either
  // a Townhall or a Peasant
  for(Integer unitID : myUnitIds)
  {
   // UnitViews extract information about a specified unit id
   // from the current state. Using a unit view you can determine
   // the type of the unit with the given ID as well as other information
   // such as health and resources carried.
   UnitView unit = newstate.getUnit(unitID);

   // To find properties that all units of a given type share
   // access the UnitTemplateView using the `getTemplateView()`
   // method of a UnitView instance. In this case we are getting
   // the type name so that we can classify our units as Peasants and Townhalls
   String unitTypeName = unit.getTemplateView().getName();

   if(unitTypeName.equals("TownHall"))
    townhallIds.add(unitID);
   else if(unitTypeName.equals("Peasant"))
    peasantIds.add(unitID);
   else
    System.err.println("Unexpected Unit type: " + unitTypeName);
  }

  // get the amount of wood and gold you have in your Town Hall
  int currentGold = newstate.getResourceAmount(playernum, ResourceType.GOLD);
  int currentWood = newstate.getResourceAmount(playernum, ResourceType.WOOD);

  List<Integer> goldMines = newstate.getResourceNodeIds(Type.GOLD_MINE);
  List<Integer> trees = newstate.getResourceNodeIds(Type.TREE);

  // Now that we know the unit types we can assign our peasants to collect resources
  for(Integer peasantID : peasantIds)
  {
   Action action = null;
   if(newstate.getUnit(peasantID).getCargoAmount() > 0)
   {
    // If the agent is carrying cargo then command it to deposit what its carrying at the townhall.
    // Here we are constructing a new TargetedAction. The first parameter is the unit being commanded.
    // The second parameter is the action type, in this case a COMPOUNDDEPOSIT. The actions starting
    // with COMPOUND are convenience actions made up of multiple move actions and another final action
    // in this case DEPOSIT. The moves are determined using A* planning to the location of the unit
    // specified by the 3rd argument of the constructor.
    action = new TargetedAction(peasantID, ActionType.COMPOUNDDEPOSIT, townhallIds.get(0));
   }
   else
   {
    // If the agent isn't carrying anything instruct it to go collect either gold or wood
    // whichever you have less of
    if(currentGold < currentWood)
    {
     action = new TargetedAction(peasantID, ActionType.COMPOUNDGATHER, goldMines.get(0));
    }
    else
    {
     action = new TargetedAction(peasantID, ActionType.COMPOUNDGATHER, trees.get(0));
    }
   }

   // Put the actions in the action map.
   // Without this step your agent will do nothing.
   actions.put(peasantID, action);
  }

  return actions;
 }

 public void terminalStep(StateView newstate, HistoryView statehistory) {
  System.out.println("Finsihed the episode");
 }

 public void savePlayerData(OutputStream os) {
  // this agent doesn't learn so nothing needs to be saved
 }

 public void loadPlayerData(InputStream is) {
  // this agent doesn't learn so nothing is loaded
 }

}
```

Building Units

Your townhall's are used to build new units. These include peasants, footmen, archers and more. Constructing a new unit requires that you have the necessary resources. These resources are specified in the map file. For these examples the map file is the `data/ResourceCollectionV2.xml`.

The first version of FirstAgent shown above uses only a single peasant. Lets modify the middle step method to make it build 2 more peasants. The code should be placed directly before the return statement in the middleStep method. You can also download this version of the agent here.

Note

Trying to build more peasants than this will hit the supply cap of food. If you want to experiment and try building more peasants you will need to modify the agent to build farms using peasants and a `COMPOUNDBUILD` action.

```
// Build 2 more peasants
if(peasantIds.size() < 3)
{
	// only try to build a new peasant if
	// the agent possess's the required resources
	// For a peasant that is 400 Gold
	if(currentGold >= 400)
	{
		// Get the peasant template's unique ID
		// this is how SEPIA identifies what type of unit to build
		TemplateView peasantTemplate = newstate.getTemplate(playernum, "Peasant");
		int peasantTemplateID = peasantTemplate.getID();

		// Grab the first townhall
		// this assumes there is at least one townhall in the map
		int townhallID = townhallIds.get(0);

		// create a new CompoundProduction action at the townhall. This instructs the
specified townhall
		// to build a unit with the peasant template ID.
		actions.put(townhallID, Action.createCompoundProduction(townhallID,
peasantTemplateID));
	}
}
```